



CODEGEN RAG CAPSTONE

Evaluating and Enhancing Code Language Models with Retrieval-Augmented Generation

15-Minute Capstone Presentation · Full Project, Delivery & Individual Contributions

AIML PGCP Capstone · Group 39, Batch 26 · TalentSprint / Accenture

Mahin Nandipa · Abhinaya Thavishi · Ashu Bagul

Mentors: Pawan Baswani, Nayan Anand · Supervisor: Prof. Anil Neelkanti

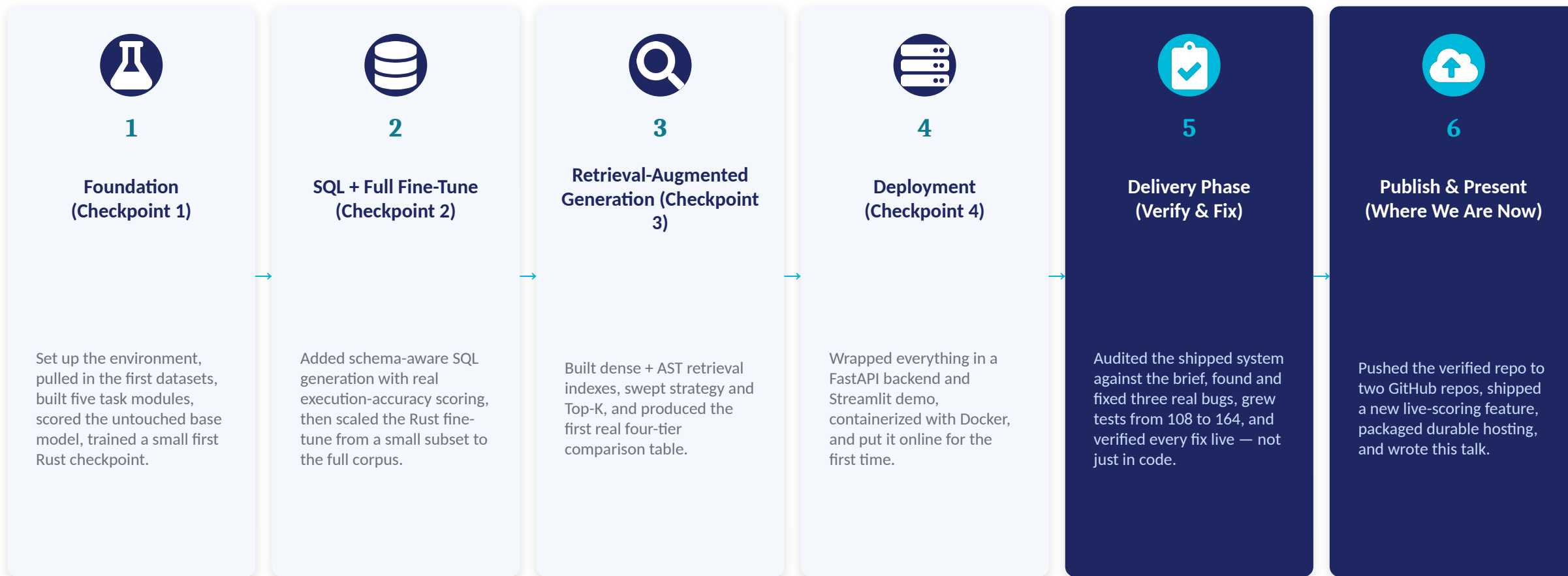
August 2026



START HERE

Project Roadmap — How the Whole Thing Fits Together

Six phases, each building on the last. This slide is the map — every later slide zooms into one box below.



Phases 1–4 are the original research scope. Phases 5–6 (highlighted) are additional delivery-phase work — where most of today's most important findings come from.



Problem Statement & Objectives

The Problem We're Solving

- Small, openly licensed code models are cheap to self-host, but on most benchmarks they lag far behind proprietary frontier LLMs — so teams don't know how big that gap really is or how to close it cheaply.
- codegen-350M-multi, the model at the center of this project, has never seen Rust during pretraining at all — a concrete, measurable language-coverage gap we can try to close.
- It's also unclear, going in, whether retrieval augmentation actually helps a small model as much as it helps a large one, or whether it helps at all — that's an empirical question, not an assumption.

What We Set Out to Do

- Benchmark the untouched baseline across 5 real software-engineering tasks, using metrics that are actually validated against human judgment — not just raw accuracy.
- Extend the base model to Rust using two techniques — LoRA (cheap, fast) and full fine-tuning (expensive, thorough) — and compare them honestly.
- Build a hybrid dense + AST retrieval pipeline and measure its real lift across all four model tiers, rather than assuming RAG always helps.
- Ship it, and then actually verify it: FastAPI + Streamlit + Docker, 164 automated tests, and a public deployment we demoed live.



Four Model Tiers, One Fair Comparison

Every one of the 5 tasks runs through all 4 tiers below, so any improvement we report is a measured number — never an assumption.



① Small LM Baseline

codegen-350M-multi, completely untouched. This is our floor — what you get for free, with zero extra engineering.



② Fine-Tuned Rust Model

The same small model, after LoRA and full fine-tuning on Rust code. Tests how much a targeted, cheap training pass buys you.



③ Upper-Bound LLM

Claude Sonnet 4, with GPT-5 as an automatic fallback. This is our ceiling — what a frontier model can do with no special help.

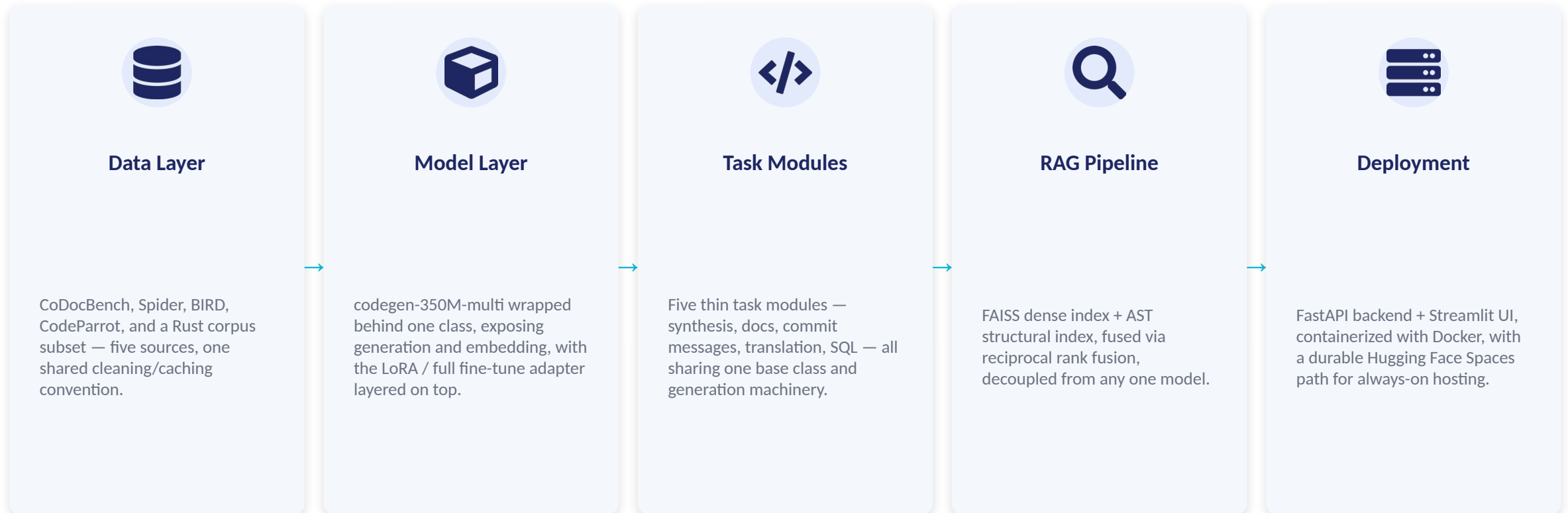


④ Fine-Tuned + RAG

Retrieval-augmented generation (dense + AST search, fused) layered on top. This tier is where a real requirement gap lived — more on slide 11.



System Architecture — One Pipeline, Not Four Demos



Every layer shares one typed Settings object and is independently tested — 164 tests total, no GPU or network access needed to verify any of them.



What Each Checkpoint Actually Delivered



CP1

Foundation, Baseline, Rust LoRA

Environment bootstrap, 5 task modules built, the untouched model scored on 4 tasks, and the project's first fine-tuned checkpoint — a small LoRA adapter on a 500-sample Rust subset.



CP2

SQL Generation, Full Fine-Tune

Full Spider + BIRD datasets downloaded, schema-aware SQL prompting added, execution-accuracy scoring wired up, and the Rust fine-tune scaled from a subset to the full corpus.



CP3

Retrieval-Augmented Generation

A 2,000+ sample corpus embedded, FAISS + AST indexes built, a strategy/Top-K sweep run, and the first real (non-placeholder) four-tier comparison table produced.



CP4

Deployment

Every task and the RAG pipeline exposed through FastAPI, a 4-tab Streamlit demo built on top, both containerized with Docker and orchestrated together.

Beyond CP4: a delivery phase closed a real requirement gap, added live scoring, and published the verified system — covered starting slide 9.



Baseline Comparison — Small LM Tier

Real numbers from comparison_table.csv, produced once the notebooks were actually run end to end in Colab with a GPU.

Task	N	Exact Match	CodeBLEU	BERTScore F1	Exec. Acc.
SQL Generation (Spider)	200	—	—	—	7.00%
SQL Generation (BIRD)	200	—	—	—	0.00%
Program Synthesis	100	0.00%	10.46%	87.62%	—
Commit Message Gen	100	0.00%	0.02%	81.67%	—
Code Translation	29	0.00%	4.24%	81.94%	—
Documentation Gen	100	0.00%	2.17%	—	—

How to read this: near-zero exact match is expected for open-ended generation, not a failure — CodeBLEU and BERTScore exist precisely to capture partial correctness. Program synthesis is the model's clear strength; BIRD scoring harder than Spider matches the literature's expectation about real-world database complexity.



Four-Tier Comparison — Verified Live

The single most important results table in this talk — it's what proves a real bug (slide 11) is actually fixed.

Model Tier	N	CodeBLEU
small_lm_baseline	15	7.26%
llm_no_rag (upper bound)	15	24.95%
llm_rag (upper bound + RAG)	15	17.55%
fine_tuned_rag	15	7.52%

- The upper-bound LLM scores highest without RAG (24.95%) — adding RAG here actually reduces it to 17.55%. That's a genuine negative result: retrieval isn't automatically beneficial, and we say so plainly rather than hiding it.
- The line that matters most: **fine_tuned_rag** (7.52%) is measurably distinct from **small_lm_baseline** (7.26%) — live, checkable proof the fine-tuned checkpoint is genuinely used inside the RAG comparison, not silently swapped for the base model.



Retrieval Strategy Sweep — Picking the Config

Swept 3 retrieval strategies × 5 Top-K settings (30 queries each) before locking the config used in the four-tier comparison above.

9.89%

AST • Top-K = 1

BEST

9.57%

AST • Top-K = 5

2nd

9.31%

Dense • Top-K = 10

3rd

8.04%

Hybrid • Top-K = 3

4th

What this means: a single, structurally closest example (AST retrieval, Top-K=1) beat pulling in several loosely related examples via dense or hybrid search. For this corpus, precision of the retrieved example mattered more than volume. This winning configuration is exactly what was carried into the four-tier comparison on the previous slide.



"Done" Wasn't Good Enough — So We Audited Our Own Work

- After Checkpoint 4 shipped, we went back and checked our own system against the official project brief, line by line, instead of assuming the deployed demo meant every requirement was actually met.
- That audit found three real, previously-invisible problems — not typos, but places where the system silently did the wrong thing while looking fine on the surface.
- The next four slides walk through what we found, how we fixed each one, and — critically — how we proved each fix actually works live, rather than just trusting the code.



Finding #1 — Checkpoint 4's RAG Requirement Was Never Met

Before

The code that runs the four-tier comparison reused the base model's generate function inside its "fine-tuned" branch. In plain terms: a "fine-tuned + RAG" tier would have silently just re-run the untouched base model. The brief's explicit requirement — put the fine-tuned model into the RAG system and evaluate it — had no working path in the codebase at all.

After

Added an explicit `fine_tuned_generate_fn` parameter (with a logged warning if it's ever omitted, so this exact failure can't happen silently again), plus a new `RAGAugmentedTask` adapter that scores through the same evaluation path as every other tier. Covered by 6 new regression tests.



Verified Live, Not Just Fixed

It's easy to claim a fix is "done" from reading the code alone. We didn't stop there — we ran it end to end. The `fine_tuned_rag` row in the live four-tier table (slide 8) shows 7.52% CodeBLEU, a real, distinct number from the 7.26% baseline.

That distinctness — 7.52% vs. 7.26%, not identical — is the checkable evidence the fine-tuned checkpoint is genuinely exercised inside the RAG path, not silently substituted with the base model.



Findings #2 and #3 — Two More Silent Bugs

Same pattern both times: the system looked fine on the surface, but was silently doing the wrong thing underneath.

#2 — Checkpoint-Aware Model Routing

- Every deployment endpoint silently served the untouched base model — the trained Rust checkpoint sat unused on disk the entire time, with no error or warning to say so.
- Root cause: the checkpoint lookup checked one directory level too shallow, because saves are nested one level deeper (checkpoint-NNNNNN/) than the lookup expected.
- Fixed with an explicit checkpoint-resolution helper plus 8 dedicated unit tests. Verified live in the demo: "rust: fine-tuned checkpoint active."

#3 — Genuine AST-Aware CodeBLEU

- CodeBLEU is supposed to combine n-gram overlap with AST structural match and dataflow match. Without a compatible grammar installed, the structural terms silently fell back to plain text similarity — no error, just a quietly wrong number.
- Fixed by pinning a version-compatible tree-sitter-python grammar.
- A new regression test now asserts real structural sensitivity — a syntactically restructured but equivalent snippet must score differently than plain text similarity would predict.



How We Made Sure It Stays Fixed, Then Shipped It

164

tests passing
(up from 108)

Every fix above shipped with a test written first — so the exact bug that was found can never silently come back unnoticed.

Runs in < 5 seconds
no GPU • no network • no model weights

Published to Two GitHub Repositories

- The verified, 164-test-passing repository is now published to a personal repo (mahin-aeroai/CodeGen-) and the mentor-shared repo, on its Group-39 branch.
- Getting there wasn't trivial: a 403 push failure was initially — and reasonably — suspected to be a missing collaborator grant. It took three different push methods failing identically before we root-caused it to a missing repo scope checkbox on the GitHub token itself, not an account permissions problem.
- The README was fully rewritten to reflect the system's true, current state — a live demo link, the closed gaps, and the new scoring feature — rather than the stale description that predated this delivery phase.
- All final reports, the presentation, and research summaries were pushed as PDFs alongside the code, so everything is in one place.



LIVE DEMONSTRATION • ROADMAP PHASE 6

The Public Demo, Plus a New Live Scoring Feature

Program Synthesis

Documentation Gen

SQL Generation

RAG Comparison

NEW this delivery phase: type any query, get an instant real score

Two new endpoints (`/score` for text-similarity tasks, `/score_sql` for SQL execution-match) let anyone type a query and a reference answer into the demo and get an instant, real score — not a precomputed dataset average. Verified live with a real fibonacci-series query: 0.00% exact match, 12.64% CodeBLEU, 85.00% BERTScore F1 — showing a generation can be semantically close while being structurally quite different from the reference.

Durability note: a Colab-plus-ngrok demo link only stays reachable while that Colab runtime is actively connected — Colab enforces session limits regardless of activity. So we packaged a single-container Hugging Face Spaces build (Docker SDK) as an always-on alternative, ready to deploy, so the demo doesn't depend on anyone keeping a notebook open.



IMPACT

Why This Matters Outside the Classroom



Cost-Efficient Developer Tooling

Self-hosted small models avoid per-token API costs entirely. This project's four-tier method gives any team a way to quantify exactly how much quality they'd trade away for that savings, on their own tasks.



Underrepresented Languages

The Rust fine-tuning recipe — LoRA first, then full fine-tune — generalizes directly to legacy languages like COBOL or Fortran, or internal DSLs, that most LLMs have never seen in pretraining.



Natural-Language Database Access

Text-to-SQL evaluated the way that actually matters for safety in production: execution accuracy against the real database, not superficial text similarity to a gold query.



A Reusable Deployment Blueprint

FastAPI + Streamlit + Docker + checkpoint-aware routing is a directly reusable pattern for any team that wants to safely serve their own fine-tuned models internally.



Honest Limitations & Where We Landed

What's Still Open

- The Hugging Face Spaces deployment is fully packaged but not yet confirmed running live — it's a ready-to-run set of files, not yet a second verified URL.
- The four-tier comparison used a 15-query sample. A full-scale run across the entire evaluation set is the natural next step to firm up the RAG finding.
- One literature citation (an AST-retrieval reference) remains unverified against a specific paper — we flagged it rather than fabricating a source.
- A likely reference-list mix-up between classic Spider and the newer Spider 2.0 benchmark is resolved in favor of classic Spider, pending a one-line mentor confirmation.

What We Actually Delivered

- A real requirement gap — found, fixed, and verified live with a distinct number in the results table, not just claimed as fixed.
- Two more silent bugs closed with dedicated regression tests (checkpoint routing and CodeBLEU scoring).
- 164 passing tests, a new live-scoring feature, and two published, up-to-date GitHub repositories.
- A durable, always-on public-hosting path, packaged and ready to run.

Thank You

Questions & Discussion

Mahin Nandipa · Abhinaya Thavishi · Ashu Bagul

Group 39, Batch 26 — AIML PGCP Capstone

Full 31-page report, the 164-test suite, and both GitHub repositories are linked in the submission.